Lecture 05.05: Types of Constructors & Overloading Simulation

📌 Executive Summary

Default vs parameterized constructors, why Python lacks native constructor overloading, and simulating multi-constructors via default arguments and @classmethod.

🔬 Theoretical Foundations & Principles

1. Python Constructor Constraints

In languages like Java or C++, multiple constructors can be declared with distinct parameter signatures. In Python, methods are bound by name within class namespaces, so declaring multiple `def __init__` simply overwrites the previous definition.

2. Simulation Patterns

1. **Default & Optional Parameters:** Use `None` defaults inside `__init__`:

```
def __init__(self, name, age=None, address=None): ...
```

2. **Factory Class Methods:** Provide alternate named constructors:

```
@classmethod
def from_string(cls, data_str): ...
```

💻 Production Implementation & Code

```
class Date:
    # 1. Primary constructor with default parameters
    def __init__(self, year: int, month: int = 1, day: int = 1):
        self.year = year
        self.month = month
        self.day = day

    # 2. Alternative constructor via @classmethod
```

```
    @classmethod
    def from_iso_string(cls, date_str: str):
        y, m, d = map(int, date_str.split("-"))
        return cls(y, m, d)

d1 = Date(2026)
d2 = Date(2026, 10, 16)
d3 = Date.from_iso_string("2026-12-25")

print(f"d1: {d1.year}-{d1.month}-{d1.day}")
print(f"d2: {d2.year}-{d2.month}-{d2.day}")
print(f"d3: {d3.year}-{d3.month}-{d3.day}")
```

⚙️ Algorithmic Complexity & Systems Analysis

- **Time Complexity:** Method dispatch overhead is $\mathcal{O}(1)$ via CPython descriptor resolution.
- **Auxiliary Space:** Instance attribute memory allocation in `__dict__` vs `__slots__`.